

Introducción a los Shaders

Por [AV8](#), Agosto 2026

<https://av8visual.vercel.app/>

Este documento es una introducción al mundo de los shaders. Su objetivo es presentar los conceptos fundamentales utilizando un lenguaje sencillo y accesible.

Aunque veremos algunos fragmentos de código, el interés principal no está en la programación, sino en comprender la lógica de los shaders: qué son, cómo funcionan y cuáles son las ideas principales detrás de ellos. Para quienes deseen profundizar en los aspectos de programación, a lo largo del documento se incluyen algunas referencias y recursos adicionales.

Se recomienda leer el documento en orden, ya que los conceptos se presentan de forma progresiva y muchos de ellos servirán de base para comprender las secciones siguientes.

- [Una breve historia de los shaders p. 1](#)
- [¿Cómo funcionan los shader? p. 4](#)
- [ISF \(Interactive Shader Format\) p. 7](#)
- [Vertex Shader p. 8](#)
- [Fragment Shader p. 13](#)
- [Referencias p. 20](#)

Una breve historia de los shaders

Para entender qué son los shaders, primero conviene diferenciar dos ideas distintas.

Por un lado, el **shading** (sombreado) como concepto, la pregunta de cómo calcular el color y/o la iluminación de una superficie en informática gráfica. Por el otro, el **shader** como un programa que se ejecuta en la GPU para generar un resultado visual, imagen o animación. Concepto y programa.

Es importante hacer esta diferencia porque el concepto de *shading* apareció mucho antes de los *shaders* programables que conocemos hoy.

Durante los años 60 y comienzos de los 70, los gráficos por computador estaban formados principalmente por modelos en líneas, tipo *wireframe*. No existían materiales, sombras o iluminación. A medida que la computación gráfica comenzó a utilizarse en investigación, ingeniería y, posteriormente, en la industria del cine, surgió la necesidad de representar los objetos con una apariencia más convincente.

Como respuesta aparecieron los primeros algoritmos de sombreado, entre ellos el **Flat Shading**, **Gouraud Shading** y **Phong Shading**. Estos algoritmos permitían calcular el color de una superficie según la dirección de la luz y la orientación del objeto. Sin embargo, todavía no eran programas independientes, sino que estaban directamente incorporados en los programas de render.

Hacia mediados de los años 80, la industria del cine comenzó a necesitar materiales mucho más variados: piel, metal, madera, agua, vidrio, humo, entre otros. Los renderizadores de la época ya incluían algunos materiales predefinidos, pero cuando era necesario crear uno nuevo, los desarrolladores debían modificar el software. Esto hacía que el sistema fuera poco flexible y difícil de ampliar.

Pixar propuso una idea diferente: en lugar de que el renderizador conociera todos los materiales posibles, este debía simplemente ser capaz de ejecutar pequeños programas escritos para describir los materiales que se necesitaran en específico. Así nació **RenderMan**, presentado en 1988, uno de los primeros sistemas en incorporar un lenguaje de shaders, que tuvo una enorme influencia en el posterior desarrollo de la informática gráfica. En ese momento todavía no existían las GPU programables; todos los cálculos se realizaban en la CPU.

Llegados los años 90, la industria de los videojuegos impulsó el crecimiento del 3D, quedando la CPU corta para procesar toda la información gráfica. Para solucionar este problema comenzaron a aparecer los aceleradores gráficos que con el tiempo evolucionaron hacia el desarrollo de las GPU modernas.

Estas primeras tarjetas gráficas utilizaban una **arquitectura de función fija** (*Fixed Function Pipeline*). Esto significa que el hardware ya venía preparado para realizar determinadas tareas gráficas, como transformar vértices, aplicar iluminación básica, texturas, niebla o mezclas de color (*blending*). Sin embargo, no se podía modificar la forma en que estos

cálculos se realizaban; únicamente se podía activar, desactivar o configurar las funciones que el fabricante había incorporado en la tarjeta.

A finales de los años 90, NVIDIA popularizó el término **GPU** con la **GeForce 256** (1999), considerada por la propia compañía como la primera *Graphics Processing Unit*. Poco después, con tarjetas como la **GeForce 3** (2001), comenzaron a aparecer de forma comercial las primeras GPU con soporte para shaders programables.

Uno de los cambios más importantes llegó en el año 2000 con **DirectX 8**, un conjunto de bibliotecas desarrollado por Microsoft para crear videojuegos y aplicaciones multimedia en Windows. DirectX 8 introdujo oficialmente los **Vertex Shaders** y los **Pixel Shaders**, permitiendo por primera vez que se pudieran escribir pequeños programas para controlar cómo la GPU transforma los vértices y calcula el color de los píxeles. Luego de eso, las nuevas generaciones de tarjetas gráficas comenzaron a incorporar estas capacidades.

Hasta ese momento, gran parte del aspecto final de una imagen estaba determinado por funciones fijas definidas por el fabricante de la tarjeta gráfica. Con la llegada de los shaders programables, muchas de esas decisiones pasaron a escribirse mediante código. En lugar de limitarse a utilizar los efectos disponibles, se hizo posible el poder crear materiales, iluminaciones y efectos visuales propios y/o personalizados.

Para que esto funcionara, era necesaria una forma de comunicación de los programas hacia la tarjeta gráfica.

Aquí aparece un concepto de informática importante: una **API** (*Application Programming Interface*). Una API puede entenderse como un conjunto de herramientas y reglas que permite que un programa se comuniquen con otro, o en este caso con la tarjeta gráfica, sin tener que conocer cómo funciona internamente cada modelo.

Durante muchos años existieron dos grandes APIs para la informática gráfica.

La primera, **DirectX**, desarrollada por Microsoft principalmente para Windows, que utiliza el lenguaje **HLSL** (*High Level Shader Language*).

La segunda, **OpenGL** (*Open Graphics Library*), un estándar abierto para programación gráfica que utiliza el lenguaje **GLSL** (*OpenGL Shading Language*).

Para mayor claridad, *OpenGL* es la API, el conjunto de herramientas que permite que un shader programable se comuniquen con la tarjeta gráfica. *GLSL* es el lenguaje con el que escribimos el shader programable.

OpenGL fue creada en 1992 por la compañía Silicon Graphics (SGI). Actualmente es mantenida por **Khronos Group**, un consorcio internacional encargado de desarrollar y mantener diferentes estándares abiertos para la informática gráfica.

GLSL permite escribir shaders que pueden utilizarse en diferentes contextos. Numerosos programas orientados al arte digital y/o la creación audiovisual permiten trabajar con shaders escritos en GLSL, entre ellos TouchDesigner, Resolume, Processing, openFrameworks o incluso se pueden integrar en la web mediante WebGL.

También es un estándar multiplataforma, por lo que los shaders escritos en GLSL pueden utilizarse en **Windows**, **Linux** y también en **macOS**. Aunque Apple actualmente desarrolla su propia tecnología gráfica, llamada **Metal**, sigue siendo posible trabajar con GLSL mediante aplicaciones compatibles.

Con el paso de los años, las GPU continuaron evolucionando para ofrecer cada vez más capacidad de programación. Se añadieron nuevas etapas programables dentro del *pipeline* gráfico, como los **Geometry Shaders**, **Tessellation Shaders**, **Compute Shaders** y, más recientemente, los **Mesh Shaders**. Al mismo tiempo aparecieron nuevas técnicas de renderizado como **PBR** (*Physically Based Rendering*), **HDR** (*High Dynamic Range*), **Deferred Rendering** y/o el **Ray Tracing** en tiempo real.

Hoy en día existen varios lenguajes de shaders y/o versiones de estos, entre ellos **GLSL 1 a 4**, **HLSL**, **WGSL** (utilizado por WebGPU) y **Metal Shading Language** (utilizado por la API Metal de Apple). Aunque pertenecen a ecosistemas diferentes, todos comparten prácticamente la misma lógica. Cambian algunas palabras, la forma de declarar ciertos elementos y algunos aspectos propios de cada plataforma, pero las ideas fundamentales son las mismas. A veces puede que una variable se declare de una forma, a veces de otra, pero en el fondo es lo mismo o muy parecido.

Finalmente, para resumir en lo esencial, la idea de shading evolucionó desde algoritmos matemáticos para calcular el color de una superficie hasta convertirse en pequeños programas que se ejecutan directamente en la GPU.

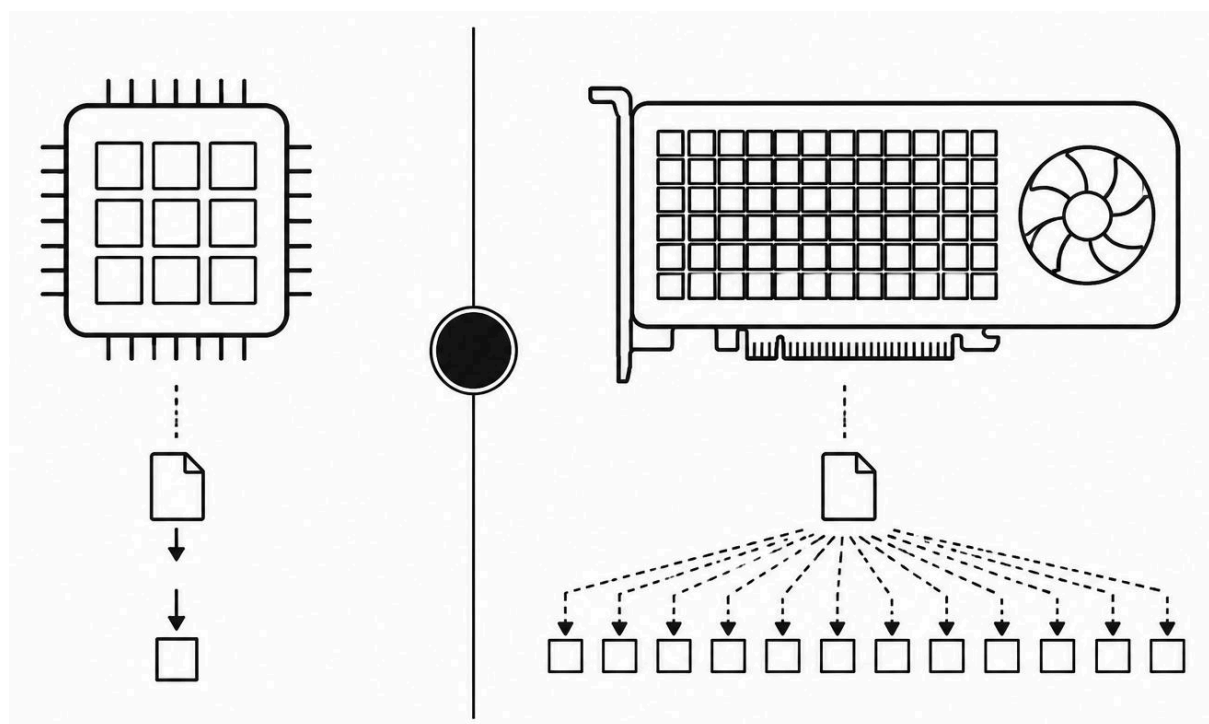
Referencias:

- <https://openglbook.com/>
- <https://www.khronos.org/>
- <https://www.aswf.io/>

¿Cómo funcionan los shaders?

Los shaders se ejecutan en la GPU (Graphics Processing Unit), un procesador especializado en el cálculo de gráficos.

A diferencia de la CPU, la GPU cuenta con miles de núcleos más simples, capaces de ejecutar el mismo programa sobre una gran cantidad de datos en paralelo. La CPU, en cambio, dispone de pocos núcleos, pero mucho más potentes y versátiles. Está diseñada para ejecutar programas de propósito general, coordinar el funcionamiento del sistema y resolver tareas que requieren instrucciones diferentes y decisiones complejas. Mientras la CPU coordina el funcionamiento general de la aplicación, la GPU realiza en paralelo los cálculos necesarios para generar la imagen.

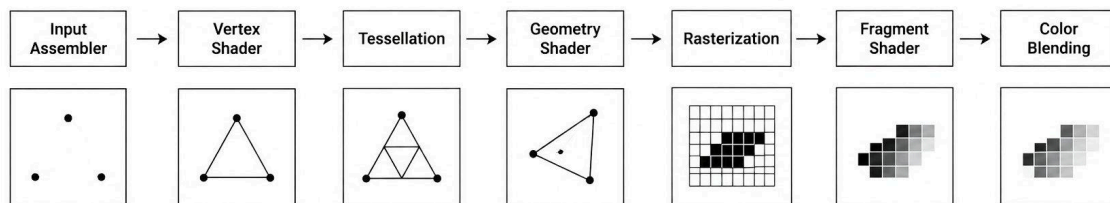


Los shaders se comunican con la GPU por medio de la API (*OpenGL*), que compila el código para que la GPU lo ejecute. Le envían información sobre la forma, los vértices, su posicionamiento en el espacio y cómo están conectados, además de información sobre el color, la textura, el material y la superficie que posteriormente será representada en píxeles en la pantalla.

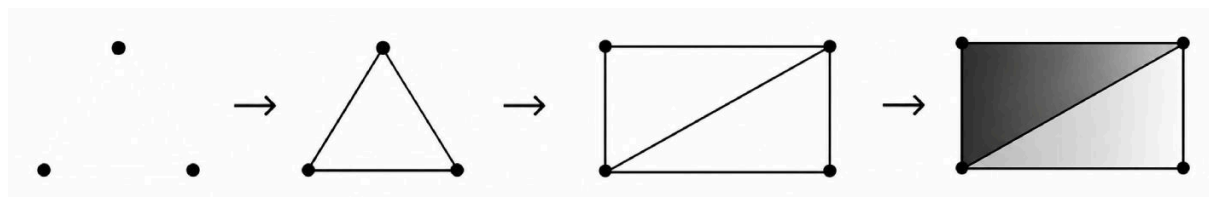
Para procesar esta información, el shader no se ejecuta una sola vez, sino muchas veces, ya sea porque parte de la información se procesa de forma independiente o porque para generar cada frame se necesita una nueva ejecución. La GPU invoca al shader cada vez que lo necesita. Eso significa que lo que se está viendo en pantalla es lo que la GPU está procesando en ese momento, y no una información guardada en un archivo de video, por ejemplo. En este sentido, el shader no tiene memoria de lo que pasó antes ni puede ir hacia adelante; trabaja únicamente con la información disponible en ese momento. Si se quieren reutilizar los frames para generar un efecto, como un delay, se pueden crear buffers de

memoria, o si se quiere guardar el resultado como video, toca grabar o capturar los fotogramas con otro proceso o aplicación.

El proceso de dibujo de cada frame está dividido en varias etapas, conocidas en su conjunto como el *pipeline* gráfico. En los sistemas gráficos modernos existen diferentes tipos de shaders que participan en este pipeline, como los *geometry shaders*, *tessellation shaders* o *compute shaders*. Sin embargo, nosotros vamos a trabajar principalmente con dos tipos de shaders: el **vertex shader** y el **fragment** o **pixel shader**. Todos los programas funcionan, como mínimo, con estos dos shaders y muchas veces ni siquiera existe la opción de utilizar los otros. Entonces, en un sentido práctico, estos son los dos tipos de shaders que nos interesan.



El **vertex shader** es el que maneja la información del espacio: las coordenadas y la posición de los vértices que van a formar la geometría sobre la cual se dibujará la superficie. Las formas se construyen a partir de triángulos, que son la unidad básica con la que la GPU construye la geometría; siempre la unión entre tres puntos. Entonces, para generar una superficie rectangular clásica se necesitan dos triángulos, y así sucesivamente, dependiendo de la complejidad de la forma. Se utilizan triángulos porque con tres puntos se tiene la seguridad de que siempre pertenecen al mismo plano; con cuatro o más puntos esto no necesariamente ocurre, lo que podría generar errores. Los modelos 3D están formados por listas de vértices y por la información de cómo están conectados entre sí. Esa información es pasada al vertex shader para poder aplicar posteriormente un **fragment** o **pixel shader** al modelo 3D. El vertex shader responde a : **¿dónde queda cada vértice?**



Una vez definida la geometría, se pasa al **fragment** o **pixel shader**, que contiene la información sobre el color y la textura de esa superficie. Se le llama *fragment* porque un fragmento es la información que se genera a partir del procesamiento del shader, mientras que un píxel es el resultado final que aparece en la pantalla. A veces un fragmento se refleja en un píxel y otras veces no, porque puede haber transparencia o quedar oculto dentro de la imagen. Por eso, el nombre técnicamente correcto es **fragment shader**, aunque mucha gente también lo conoce como pixel shader. El fragment shader responde a : **¿de qué color será cada píxel?**

La generación de imágenes mediante shaders se basa principalmente en el álgebra lineal. Esta permite representar, posicionar y transformar la geometría en el espacio, así como realizar operaciones sobre el color y otros atributos visuales, mediante vectores, matrices y

funciones matemáticas, como las trigonométricas. No hace falta ser un experto, pero es bueno saberlo si se quiere entender un poco más de cómo se genera la imagen.

Lo principal a tener claro son el **vertex shader** y el **fragment shader**, que son los scripts, o archivos de texto, en los que vamos a escribir el programa que será ejecutado. El vertex shader se encarga principalmente de la geometría, aunque también puede manejar información de color, y el fragment shader calcula el valor final de los píxeles. Ambos trabajan de manera coordinada para generar cada imagen. Entonces, siempre vamos a necesitar de dos scripts diferentes, el vertex y el fragment; aunque hay casos en los que el software solo permite modificar el fragment shader, porque el vertex shader ya viene definido por defecto. Ese es el caso de *Resolume*, por ejemplo. *Resolume* asume que el resultado final siempre será el rectángulo de salida del layer donde se ejecuta el shader y, por esa razón ya viene con el vertex shader rectangular pre-establecido, y únicamente permite cargar el fragment shader.

El texto puede tener algunas imprecisiones técnicas avanzadas en aras de la intuición.

ISF (Interactive Shader Format)

El **ISF (Interactive Shader Format)** es un **formato** que se creó para que un mismo shader pudiera ejecutarse en distintos programas sin tener que reescribirlo para cada uno. La idea fue estandarizar una forma de escribir shaders para que funcionaran en varios programas sin necesidad de adaptarlo, en la mayoría de los casos. Este formato fue desarrollado en 2013 por **Vidvox**, desarrolladores del software *Vdmx*, pensado originalmente para ese programa, luego fue adoptado por otros.

Hoy en día el formato es compatible con software como *Resolume* (via *Wire*), *MadMapper*, *Smode*, *Millumin* entre otros. *Vidvox* también desarrolló un editor, visualizador, práctico y multiplataforma llamado **ISF editor** proveyendo una forma sencilla de testear y ajustar los ISF shaders.

Un shader ISF sigue siendo, en esencia, un shader GLSL, pero tiene algunos ajustes, como un bloque en formato JSON, en el que se hace una descripción y se declaran los inputs que queremos estén disponibles para manipular la imagen, además de algunas variables ya definidas de forma automática (*built-in*) y ciertas convenciones propias del formato.

Una de las ventajas de declarar los inputs en el bloque JSON es que se vuelven accesibles directamente en la interfaz del programa como sliders, botones, pads u otros controles, según el tipo de input que se haya definido.

Gracias a las tecnología de ai de hoy en día, resulta bastante simple pasar de un shader clásico GLSL a un ISF.

Referencias

- <https://isf.vidvox.net/desktop-editor/>
- <https://isf.video/>
- <https://editor.isf.video/>
- <https://www.vidvox.net/>

Vertex Shader

En esta sección revisaremos dos **vertex shaders** escritos en **GLSL ES 1.00** (WebGL 1 / OpenGL ES 2.0). A medida que los analizamos, iremos explicando los conceptos fundamentales del lenguaje.

```
src > shaders > ≡ vertex.glsl
1  uniform mat4 projectionMatrix;
2  uniform mat4 viewMatrix;
3  uniform mat4 modelMatrix;
4
5  attribute vec3 position;
6  attribute vec2 uv;
7
8  varying vec2 v_uv;
9
10 void main() {
11     v_uv = uv;
12
13     gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.);
14 }
15
```

uniform mat4 projectionMatrix;
uniform mat4 viewMatrix;
uniform mat4 modelMatrix;

uniform : los **uniforms** son variables cuyo valor se envía desde el programa al shader y permanece igual para todos los vértices o fragmentos mientras se dibuja un objeto.

mat4 : es el tipo de dato, una **matriz de 4 × 4**, una tabla de **4 filas y 4 columnas** de números, o **4 vec4**

Tipos de datos: en los shaders, todos los datos tienen un tipo definido para que la GPU pueda realizar los cálculos matemáticos de forma eficiente. Como los shaders trabajan principalmente con geometría y gráficos, sus tipos de datos están pensados para el álgebra lineal.

Los tipos más comunes son:

- **int**: números enteros.
- **float**: números decimales o de punto flotante.
- **vec2**, **vec3** y **vec4**: vectores de dos, tres y cuatro componentes.
- **mat2**, **mat3** y **mat4**: matrices de 2×2, 3×3 y 4×4.
- **bool**: valores booleanos, que solo pueden ser **true** o **false** (verdadero o falso).

En la práctica, los **int** se usan muy poco. La mayor parte del tiempo se trabaja con **float**, ya que casi todos los cálculos de posición, color, movimiento e iluminación requieren

números decimales. Los **vectores** también son uno de los tipos de datos más utilizados, porque permiten representar posiciones, direcciones, colores y coordenadas de textura.

Las operaciones matemáticas solo pueden realizarse entre tipos de datos compatibles. Si los tipos no coinciden, el shader generará un error de compilación. Por ejemplo, no es posible realizar directamente una operación entre un **int** y un **float**:

```
int x = 5;  
float y = 2.5;  
float z = x + y; // Error
```

Si es necesario combinar distintos tipos de datos, primero hay que convertir uno de ellos al tipo adecuado. Esto se conoce como conversión de tipo o *casting*.

```
float z = float(x) + y;  
o bien:  
int z = x + int(y);
```

Los vectores (**vec2**, **vec3**, **vec4**) y las matrices (**mat2**, **mat3**, **mat4**) también pueden participar en operaciones matemáticas, siempre que los tipos sean compatibles. En la práctica, casi siempre contienen valores de tipo float, por lo que es habitual trabajar con **float**, **vec** y **mat**.

; : en los shaders todas las instrucciones deben terminan con un ;. Si se omite, el shader generará un error de compilación. Olvidarlo es uno de los errores de sintaxis más comunes. La excepción son las **directivas del preprocesador** (por ejemplo, **#define** o **#ifdef**), que no utilizan punto y coma.

uniform mat4 projectionMatrix; : declara la **matriz de proyección**, que transforma la escena para que pueda verse desde la cámara y proyectarse sobre la pantalla (perspectiva u ortográfica).

uniform mat4 viewMatrix; : declara la **matriz de vista**, que representa la posición y orientación de la cámara en la escena.

uniform mat4 modelMatrix; : declara la **matriz de modelo**, que contiene la posición, rotación y escala del objeto en el mundo.

Aunque a veces toque declararlas como variables, estas matrices normalmente ya vienen predefinidas por el motor gráfico, por lo que no es necesario crearlas ni asignarles un valor manual previo.

```
5  attribute vec3 position;  
6  attribute vec2 uv;  
7  
8  varying vec2 v_uv;
```

attribute vec3 position;
attribute vec2 uv;

attribute : los **attributes** son variables que almacenan datos que pertenecen a cada vértice de la geometría. A diferencia de los **uniforms**, cuyo valor es el mismo para todos los vértices, los **attributes** pueden tener un valor diferente en cada uno. Estos datos pueden representar la *posición* de cada vértice, sus *coordenadas*, sus *normales* (el vector perpendicular a la superficie, utilizado principalmente para los cálculos de iluminación en 3D) o incluso un *color* específico para cada vértice.

attribute vec3 position; : **position** contiene la posición de cada vértice en el espacio local del objeto. Es un **vec3** porque cada posición está formada por tres componentes: **x**, **y** y **z**, correspondientes al espacio tridimensional.

attribute vec2 uv; : **uv** contiene las coordenadas de textura de cada vértice. Es un **vec2** porque está formado por dos componentes: **u** y **v**, que corresponden a un espacio bidimensional utilizado para mapear una textura sobre la geometría.

varying vec2 v_uv;

varying: los **varying** son variables que permiten enviar información desde el **vertex shader** hacia el **fragment shader**. Normalmente es información espacial o de coordenadas.

varying vec2 v_uv; : se declara la variable **v_uv**, que se utilizará para enviar las coordenadas (**uv**) desde el vertex shader al fragment shader. Es un **vec2** porque debe corresponder al tipo de dato que va a enviar.

Por convención, suele utilizarse el prefijo **a** para los **attributes**, **u** para los **uniforms** y **v** para los **varying**. Aunque no es obligatorio y no siempre encontrarás esta nomenclatura, es una buena práctica porque facilita identificar el tipo de cada variable al leer el código.

```

10 void main() {
11     v_uv = uv;
12
13     gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.);
14 }

```

void main() : es la función principal del shader, donde se escriben todas las instrucciones que debe ejecutar la GPU. En un vertex shader, esta función se ejecuta **una vez por cada vértice** de la geometría.

v_uv = uv; : aquí se copia el valor del atributo **uv** al **varying v_uv**, para enviarlo al *fragment shader*.

gl_Position : es la variable más importante del vertex shader, es el resultado final del vertex shader. En ella se escribe la posición final de cada vértice.

gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.); : esta es la forma clásica de calcular la posición final de un vértice. Primero, **position** se convierte de un **vec3** a un **vec4**, ya que las matrices utilizadas son de tipo **mat4** y ambos tipos deben ser compatibles para poder multiplicarse. Luego, las matrices **modelMatrix**, **viewMatrix** y **projectionMatrix** transforman esa posición desde el espacio local del objeto hasta su posición final en la pantalla.

projectionMatrix: proyecta la escena sobre la pantalla.

viewMatrix: aplica la posición y orientación de la cámara.

modelMatrix: coloca el objeto en la escena.

```

shaders > ≡ vertex.glsl
1  attribute vec3 a_position;
2  varying vec2 v_texcoord;
3
4  void main(void) {
5      v_texcoord = a_position.xy;
6
7      gl_Position = vec4(a_position, 1.0);
8  }

```

Este es otro vertex shader. Es muy parecido al anterior, pero no utiliza las matrices **modelMatrix**, **viewMatrix** ni **projectionMatrix**. Esto se debe a que las posiciones almacenadas en **a_position** ya vienen preparadas para escribirse directamente en **gl_Position**, por lo que no necesitan transformaciones adicionales. Dependiendo del motor gráfico o del tipo de aplicación, estas transformaciones pueden realizarse antes de que los datos lleguen al shader.

v_texcoord = a_position.xy; : aquí se copian únicamente las componentes **x** e **y** de **a_position** y se almacenan en **v_texcoord**.

a_position es un **vec3**, por lo que contiene tres componentes: **x**, **y** y **z**. Sin embargo, **v_texcoord** es un **vec2**, por lo que solo puede almacenar **dos** componentes. La notación **.xy** indica que queremos tomar únicamente esos dos valores.

Esta técnica se conoce como *swizzling*, y permite acceder o reorganizar las componentes de un vector. Por ejemplo:

- `position.x` → obtiene solo la componente **x**.
- `position.xy` → obtiene las componentes **x** e **y** como un **vec2**.
- `position.xyz` → obtiene las tres componentes como un **vec3**.
- `position.yx` → obtiene las componentes **y** y **x**, cambiando su orden.

En shaders, los vectores pueden tener hasta **cuatro** componentes. Para acceder a ellas se utilizan tres nomenclaturas equivalentes:

x, y, z, w : se usan normalmente para posiciones y coordenadas espaciales.

r, g, b, a : se usan habitualmente para representar colores.

s, t, p, q : se utilizan principalmente para coordenadas de textura.

Las tres nomenclaturas hacen referencia a las mismas componentes; solo cambia el nombre para que el código sea más fácil de leer según el contexto.

Por ejemplo, si tenemos: **vec4** dato;

Podemos acceder a sus componentes de distintas formas:

`dato.xy`

`dato.rgb`

`dato.st`

`dato.yx`

`dato.bgra`

Esta técnica se conoce como *swizzling* y permite seleccionar o reordenar las componentes de un vector. Lo único que no está permitido es mezclar nomenclaturas en un mismo acceso. Por ejemplo, `dato.xg` produciría un error.

Fragment Shader

En esta sección revisaremos dos **fragment shaders** escritos en **GLSL ES 1.00** (WebGL 1 / OpenGL ES 2.0). A medida que los analizamos, iremos explicando los conceptos fundamentales del lenguaje.

```
src > shaders > fragment.glsl
1  precision mediump float;
2
3  uniform float u_time;
4
5  varying vec2 v_uv;
6
7  void main() {
8
9      float t = sin(u_time) * 0.5 + 0.5;
10
11
12      gl_FragColor = vec4(v_uv, t, 1.);
13  }
14
```

precision mediump float; : esta línea define la **precisión por defecto** que tendrán los valores de tipo **float** en el fragment shader. Existen tres niveles de precisión: **lowp**, **mediump** y **highp**, que representan una precisión baja, media y alta, respectivamente. Hay ocasiones en las que no es necesaria ponerla.

uniform float u_time; : es una variable que almacena el tiempo transcurrido. Recordemos que los **uniforms** son variables cuyos valores se envían desde el entorno (JavaScript, Three.js, OpenGL, etc.). En el fragment shader es muy común utilizar **uniforms** para enviar información como el tiempo (**u_time**), texturas (imágenes, videos o la cámara), la resolución de la pantalla, la posición del mouse o cualquier otro valor que queramos controlar desde el programa, variables que podemos tener como sliders, botones o sensores desde nuestra interfaz, por ejemplo.

varying vec2 v_uv; : esta línea declara la variable **v_uv**, que recibe la información enviada desde el vertex shader. En este caso las coordenadas **uv**.

void main() : al igual que en el vertex shader, es la función principal del shader. Todo el código que se encuentra dentro de ella es ejecutado por la GPU. En el fragment shader, esta función se ejecuta **una vez por cada fragmento**. Mientras que el vertex shader procesa los **vértices** de la geometría, el fragment shader procesa los **fragmentos**, que posteriormente se convertirán en los **píxeles** visibles de la imagen.

float t = sin(u_time) * 0.5 + 0.5; : declara una variable llamada **t** y se le asigna un valor que cambia continuamente con el tiempo.

- **sin(u_time)** : calcula el seno del tiempo. Este valor oscila continuamente entre -1. y 1., generando un movimiento periódico.
- *** 0.5** : reduce esa amplitud a la mitad, haciendo que el rango pase de -1. a 1., a -0.5 a 0.5
- **+ 0.5** : desplaza el rango hacia arriba, obteniendo finalmente valores entre 0. y 1.

Como resultado, **t** aumenta y disminuye suavemente entre 0. y 1. a medida que transcurre el tiempo.

En shaders es muy común **normalizar** valores, es decir, transformarlos para que queden dentro del intervalo **0** a **1**. Muchos parámetros, como los colores, la transparencia, las mezclas o las coordenadas UV, utilizan este rango. A este intervalo entre **0** y **1** se le conoce como **intervalo normalizado**.

Trabajar con valores normalizados hace que los cálculos sean más consistentes e independientes de la resolución o del tamaño de la pantalla.

Por ejemplo, es habitual utilizar un sistema de **coordenadas normalizadas**, de **0.0** a **1.0**. Es posible trabajar directamente con coordenadas en píxeles, pero esto suele hacer que el shader dependa de la resolución. Si el tamaño de la ventana cambia, es necesario adaptar los cálculos. En cambio, las **coordenadas normalizadas** se ajustan automáticamente a cualquier resolución y facilitan el desarrollo de shaders reutilizables.

Los colores también se representan con valores normalizados. Cada canal del modelo **RGBA (r, g, b, a)** toma un valor entre **0.0** y **1.0**, donde **0.0** significa ausencia de ese componente y **1.0** su intensidad máxima. En el **canal alfa**, **0.0** representa una transparencia total y **1.0** una opacidad completa.

En shaders, cuando escribimos un valor de tipo **float**, el número debe incluir un punto decimal. Por ejemplo:

1.0
1.
0.7
.7

Todas estas formas representan un **float**. No es obligatorio escribir el 0 antes o después del punto cuando esa parte vale cero, pero sí es obligatorio escribir el punto decimal.

Si escribimos:

1

El programa interpreta ese valor como un **int**, no como un **float**. Esto puede producir errores al realizar operaciones entre tipos de datos incompatibles, por lo que *olvidar el punto decimal es uno de los errores más comunes al comenzar a programar shaders*.

gl_FragColor : es la variable principal del *fragment shader*, así como es el **gl_Position** para el *vertex shader*. En ella se escribe el **color final** para cada **fragmento**.

gl_FragColor = vec4(v_uv, t, 1.); : como **gl_FragColor** es un **vec4**, debemos construir un vector con cuatro componentes. En este caso:

- **v_uv** es un **vec2**, por lo que aporta las dos primeras componentes del vector.
- **t** es un float y aporta la tercera componente.
- **1.** aporta la cuarta componente.

Es equivalente a escribir: `gl_FragColor = vec4(v_uv.x, v_uv.y, t, 1.);`

Como **gl_FragColor** representa un **color**, estos **cuatro** componentes corresponden a los canales **rgba** : **rojo** (*red*), **verde** (*green*), **azul** (*blue*) y **alfa** (*transparencia*), respectivamente.

Ver resultado de este fragment shader : <https://santirockk.github.io/fsexample1/>

Ahora vamos a ver un fragment shader un poco más complejo. No es necesario comprender en detalle cómo funciona. Por ahora, lo importante es observar su estructura y hacerse una idea de cómo se construye un fragment shader más elaborado.

```
precision mediump float;

uniform float u_time;
uniform vec2 u_resolution;
varying vec2 v_texcoord;

#define PI 3.14159265359
#define TAU (PI * 2.)

vec3 blue = vec3(0.18, 0.7, 0.92);
vec3 orange = vec3(0.87, 0.49, 0.38);
vec3 green = vec3(0.3, 0.65, 0.43);
vec3 peach = vec3(0.96, 0.76, 0.55);
vec3 black = vec3(0.);

float POINTS = 24.;
float speed = 0.15;
float dotRadiusMask = 0.19;
float scale = 0.77;
float radius_a = 0.23 ;

// Shapes
float circle(vec2 p, float r) {
    return 1. - smoothstep(r, r + 0.003, length(p));
}

float ringMask(float d, float r, float w) {
    float outer = smoothstep(r + w * .5, r + w * .5 - 0.002, d);
    float inner = smoothstep(r - w * .5, r - w * .5 + 0.002, d);
    return outer * inner;
}
```



```

float dotMask(vec2 uv, float r, float w, float direction) {
    float d = length(uv);
    float angle = atan(uv.y, uv.x);
    angle += direction * u_time * speed;
    float sector = TAU/POINTS;
    float local = mod(angle + sector * .5, sector) - sector * .5;

    vec2 p;
    p.x = d - r;
    p.y = local * r;

    float dotRadius = w * dotRadiusMask;
    return circle(p, dotRadius);
}

void drawRing(
    inout vec3 color,
    vec2 uv,
    float radius,
    float width,
    vec3 ringColor,
    vec3 dotColor,
    float direction
) {
    float d = length(uv);
    float ring = ringMask(d, radius, width);
    if(ring < 0.001) return;
    color = mix(color, ringColor, ring);
    float dots = dotMask(uv, radius, width, direction);
    color = mix(color, dotColor, dots * ring);
}

void main() {
    vec2 uv = v_texcoord;
    uv.x *= u_resolution.x/u_resolution.y;

    vec3 color = blue;
    float radius = 2.;

    for(int i=0; i<21; i++) {
        bool org = mod(float(i), 2.) == 0.;

        drawRing(color,
            uv,
            radius,

```

```

        radius * radius_a,
        org ? orange : green,
        org ? black : peach,
        org ? 1. : -1.
    );
    radius *= scale;
}

gl_FragColor = vec4(color, 1.);
}

```

uniform float u_time;

uniform vec2 u_resolution;

varying vec2 v_texcoord;

uniform float u_time; la variable que almacena el tiempo transcurrido

uniform vec2 u_resolution; esta variable almacena la resolución de la ventana de salida, el *width* y el *height*, es un vector 2, a veces puede ser útil para ciertos cálculos.

varying vec2 v_texcoord; variable que recibe la información de *a_position.xy* del vertex shader.

#define PI 3.14159265359

*#define TAU (PI * 2.)*

Variables para el preprocesador, no llevan ";" al final.

vec3 blue = vec3(0.18, 0.7, 0.92);

vec3 orange = vec3(0.87, 0.49, 0.38);

vec3 green = vec3(0.3, 0.65, 0.43);

vec3 peach = vec3(0.96, 0.76, 0.55);

vec3 black = vec3(0.);

Paleta de color, no necesariamente se debe declarar de esta forma, pero a mi me gusta hacerlo así.

vec3 black = vec3(0.); si, por ejemplo, todos los 3 valores de este vector son iguales, en este caso 0., no es necesario declarar *vec3(0., 0., 0.)* con declarar *vec3(0.)* el programa asume que todos los componentes del *vec3* son 0.

float POINTS = 24.;

float speed = 0.15;

float dotRadiusMask = 0.19;

float scale = 0.77;

float radius_a = 0.23 ;

Variables que dejamos libres para intervenir, modificar, ajustar el resultado final.

Eventualmente, podemos convertirlas en *uniforms* y luego en sliders para controlar sus valores desde la interfaz.

```

// Shapes
float circle(vec2 p, float r) {
    return 1. - smoothstep(r, r + 0.003, length(p));
}

float ringMask(float d, float r, float w) {
    float outer = smoothstep(r + w * .5, r + w * .5 - 0.002, d);
    float inner = smoothstep(r - w * .5, r - w * .5 + 0.002, d);
    return outer * inner;
}

float dotMask(vec2 uv, float r, float w, float direction) {
    float d = length(uv);
    float angle = atan(uv.y, uv.x);
    angle += direction * u_time * speed;
    float sector = TAU/POINTS;
    float local = mod(angle + sector * .5, sector) - sector * .5;
    vec2 p;
    p.x = d - r;
    p.y = local * r;
    float dotRadius = w * dotRadiusMask;
    return circle(p, dotRadius);
}

void drawRing(
    inout vec3 color,
    vec2 uv,
    float radius,
    float width,
    vec3 ringColor,
    vec3 dotColor,
    float direction
) {
    float d = length(uv);
    float ring = ringMask(d, radius, width);

    if(ring < 0.001) return;
    color = mix(color, ringColor, ring);
    float dots = dotMask(uv, radius, width, direction);
    color = mix(color, dotColor, dots * ring);
}

```

Funciones (*auxiliares*) que nos van a ayudar a crear la o las formas que queremos para nuestra imagen final. Como podrán notar, la tercera función **dotMask()** utiliza la primera función **circle()**, y la cuarta función **drawRing()** utiliza la segunda y tercera función **ringMask()** y **dotMask()**, luego esta cuarta función **drawRing()** será la que llamemos en la función **main()**.

Es muy común construir funciones a partir de otras funciones. De esta manera, a partir de operaciones sencillas es posible crear formas y comportamientos cada vez más complejos. Este principio es una de las bases de la programación y también del desarrollo de shaders.

También podemos notar que estas funciones están declaradas fuera de **main()**. Esto se debe a que **main()** es la función principal del shader, mientras que las demás son **funciones auxiliares** que pueden llamarse desde **main()** o incluso desde otras funciones. Declararlas por separado hace que el código sea más claro, más fácil de reutilizar y más sencillo de mantener.

Si observamos con un poco más de detalle el interior de estas funciones, encontraremos llamadas a otras funciones como **smoothstep()**, **length()**, **atan()**, **mod()** o **mix()**. Estas forman parte de la biblioteca estándar de GLSL, por lo que ya vienen incluidas en el lenguaje y no es necesario declararlas.

Para conocer qué funciones están disponibles y cómo funcionan, recomendamos consultar el glosario de *The Book of Shaders*, una excelente referencia para aprender GLSL: <https://thebookofshaders.com/glossary/>

En este ejemplo también encontramos estructuras comunes de programación, como los bucles **for** :

```
for(int i = 0; i < 21; i++)
```

estructuras condicionales tipo:

```
if(ring < 0.001) return;
```

o de **operador condicional (ternario)**:

```
org ? orange : green
```

La idea no es profundizar en estos conceptos, ya que pertenecen a la programación en general, sino mostrar que este tipo de técnicas también se utilizan dentro de los shaders.

uv.x *= u_resolution.x / u_resolution.y; : esta línea no siempre es necesaria. Se utiliza para corregir la relación de aspecto de la pantalla, de modo que una unidad en el eje **X** tenga el mismo tamaño visual que una unidad en el eje **Y**. Esto evita que las formas se deformen cuando la ventana no es cuadrada; por ejemplo, permite que un círculo siga viéndose como un círculo y no como un óvalo.

Ver resultado de este fragment shader : <https://santirockk.github.io/fspoints/>

Referencias

- <https://thebookofshaders.com/>

Referencias :

- <https://thebookofshaders.com/>
- <https://www.shadertoy.com/>
- <https://openglbook.com/>
- <https://www.khronos.org/>
- <https://isf.video/>
- <https://isf.vidvox.net/desktop-editor/>
- <https://editor.isf.video/>
- <https://www.vidvox.net/>
- <https://www.aswf.io/>

Por [AV8](#), Agosto 2026

<https://av8visual.vercel.app/>